



RÉALISATION D'UN TRAITEMENT DANS UN ENVIRONNEMENT CLOUD BIG DATA

Grégoire Mureau

PROBLÉMATIQUE ET JEU DE DONNÉES

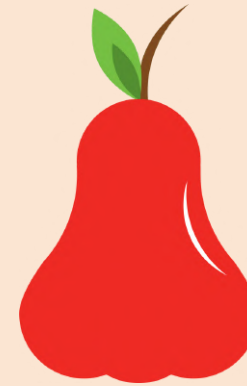




Contexte

- Objectif de Fruits! :
 - Développement robots cueilleurs intelligents
 - Préserver biodiversité des fruits
 - Traitement spécifique par espèce
- Première étape:
 - Application mobile de reconnaissance de fruits
 - Sensibilisation du grand public
 - Validation du moteur de classification





Ma mission

Reprise des travaux de l'alternant pour compléter une chaîne de feature extraction

• Contexte de départ

- Travaux incomplets sur AWS EMR
- Notebook existant avec approche transfer learning (MobileNetV2)
- Manque : broadcast des poids modèle + PCA distribuée

• Travail à réaliser

- Compréhension travaux existants
- Complétion chaîne traitement Big Data
- Implémentation PCA (réduction dimensionnalité)
- Préparation de l'architecture pour passage à l'échelle

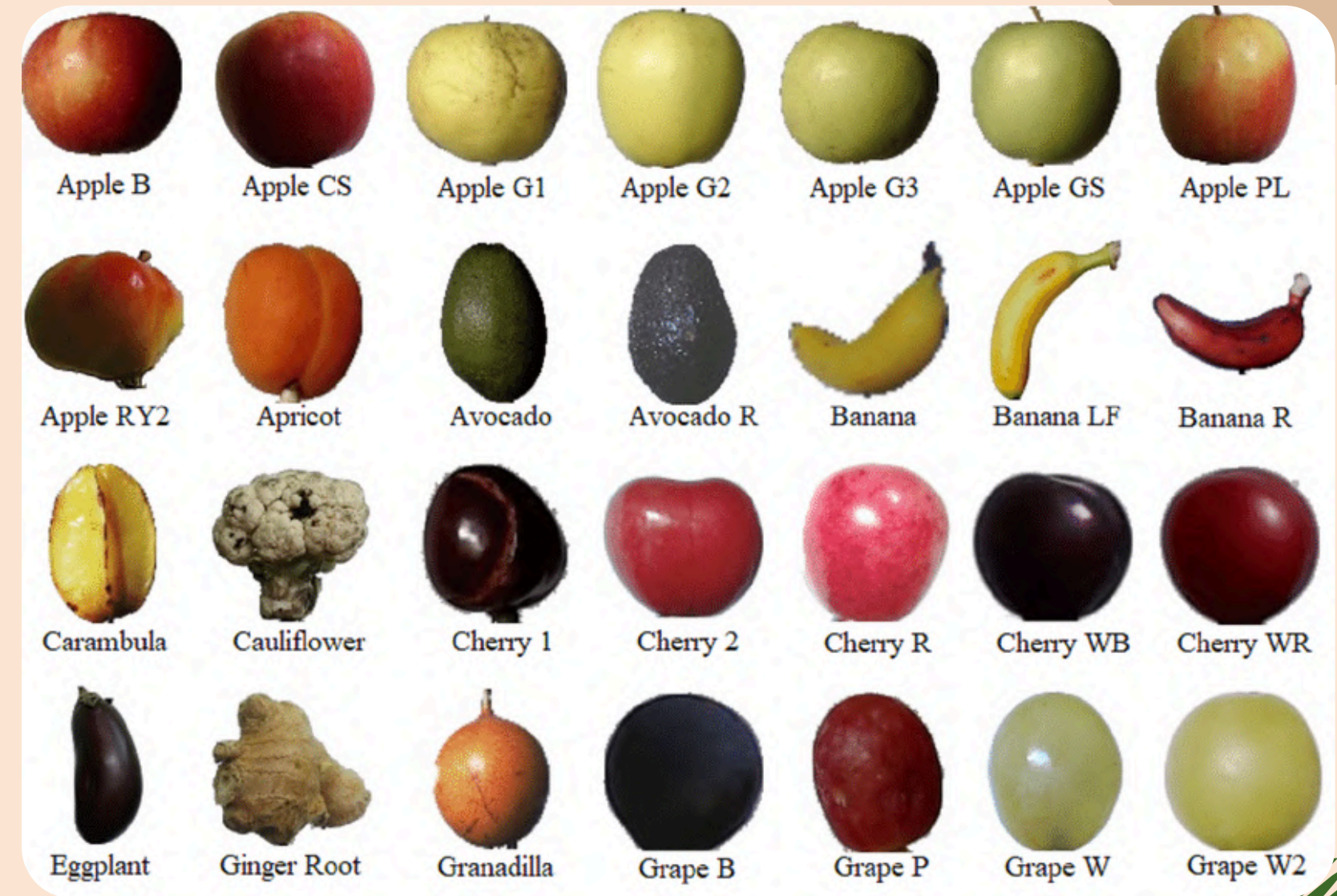
• Contraintes imposées

- Scripts PySpark obligatoires (calcul distribué)
- Conformité RGPD
- Optimisation coûts (<10€)



Jeu de données

- Fruits-360
- Composition
 - 147691 images
 - Format : 100x100 pixels, couleur
 - Test pipeline : 103 images test-multiple_fruits
- Approche de validations:
 - Développement et test en local d'abord
 - Migration vers cluster EMR cloud
 - Validation sur échantillon réduit
 - Architecture prête pour volume complet



ENVIRONNEMENT BIG DATA



Architecture Cloud AWS

- AWS:
 - Leader marché cloud computing
 - Offre Big Data complète et mature
 - Facturation à la demande
 - Services intégrés
- Service utilisés :
 - EMR : cluster calcul distribué (Spark, Hadoop)
 - EC2 : Instances de calcul sous-jacentes
 - S3 : Stockage données et résultats (artefacts)
 - Région EU-West-3 (Paris) : Conformité RGPD



Configuration Cluster EMR

- Architecture :
 - 1 Master (driver Spark)
 - 2 Workers (executors Spark)
 - Instances m5.xlarge
 - Région eu-west-3 (Paris)
- Personnalisation :
 - Bootstrap : packages Python automatiques
 - Notebook persistant sur S3
 - Configuration sécurité SSH, accessible depuis navigateur

- Logiciels installés :



```
$ bootstrap-emr.sh
1  #!/bin/bash
2  sudo python3 -m pip install --upgrade pip setuptools
3  sudo python3 -m pip install numpy==1.19.5
4  sudo python3 -m pip install pandas==1.2.5
5  sudo python3 -m pip install pillow
6  sudo python3 -m pip install pyarrow
7  sudo python3 -m pip install fsspec
8  sudo python3 -m pip install s3fs
```




Accès Sécurisé

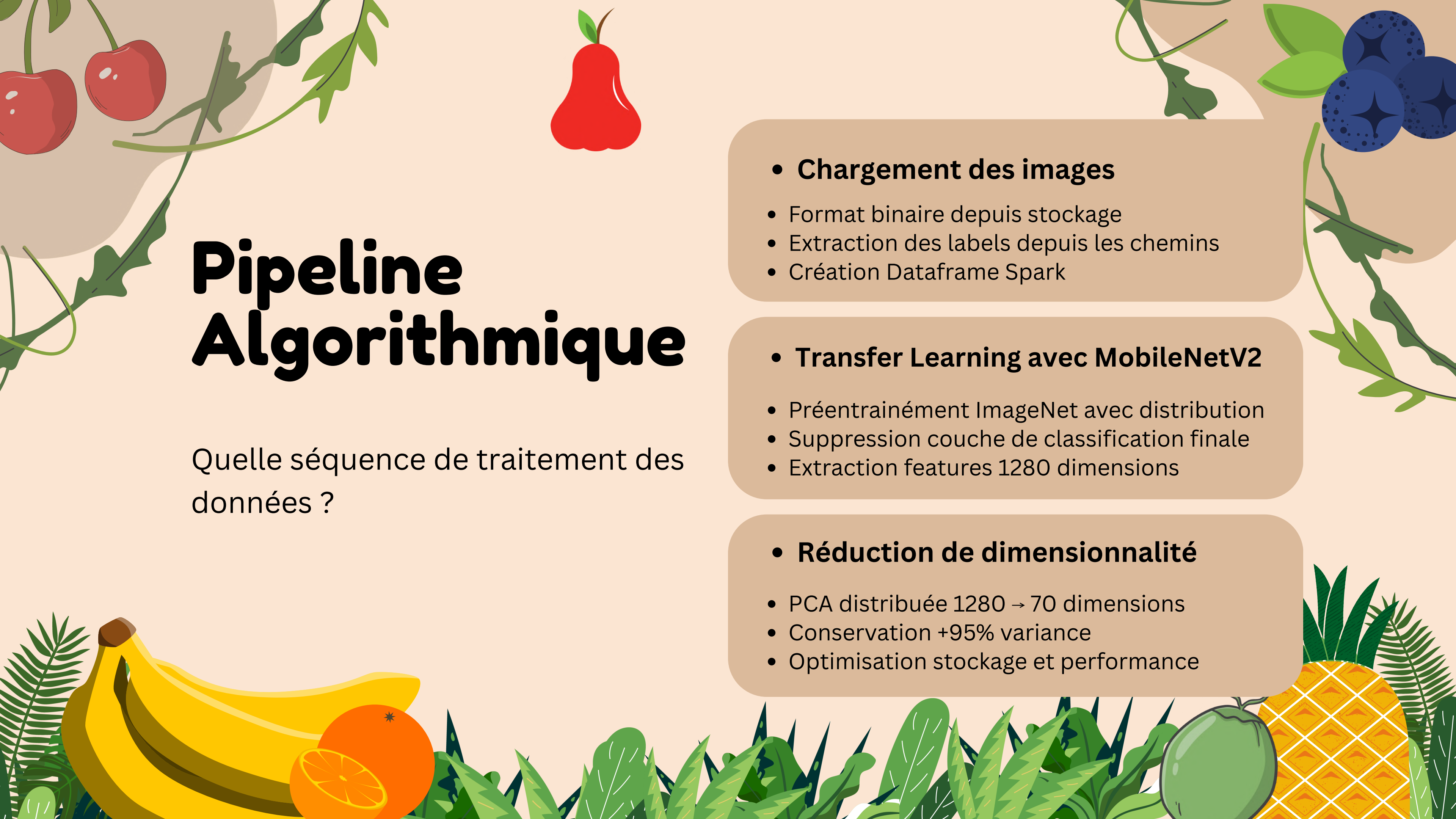
- Configuration sécurisée SSH, ouverture port 22
- Tunnel SSH avec port forwarding :

```
ssh -i ./emr-keypair-p11.pem -L 8890:localhost:8890 -L 9443:localhost:9443 hadoop@[IP-EMR]
```

- Avantages:
 - Connexion chiffrée de bout en bout
 - Pas de proxy externe nécessaire (~~FoxyProxy~~)
 - Contrôle précis des ports
 - Accès direct aux applications cluster
- Applications accessibles:
 - **JupyterHub** (9443) : développement notebooks
 - Interface monitoring cluster (8890)

CHAÎNE DE TRAITEMENT PYSPARK





Pipeline Algorithmique

Quelle séquence de traitement des données ?

- **Chargement des images**

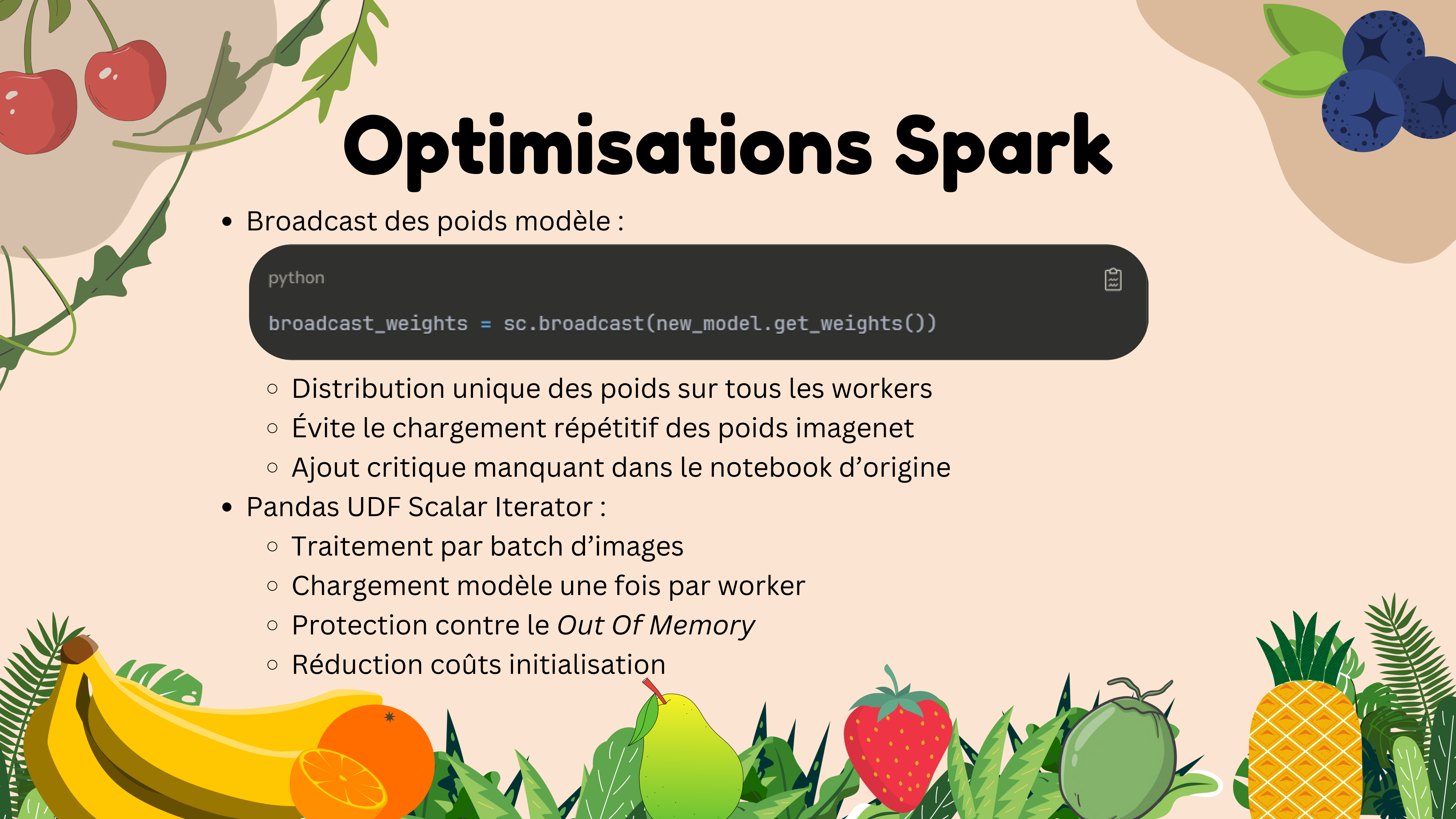
- Format binaire depuis stockage
- Extraction des labels depuis les chemins
- Création Dataframe Spark

- **Transfer Learning avec MobileNetV2**

- Préentraînement ImageNet avec distribution
- Suppression couche de classification finale
- Extraction features 1280 dimensions

- **Réduction de dimensionnalité**

- PCA distribuée 1280 → 70 dimensions
- Conservation +95% variance
- Optimisation stockage et performance



Optimisations Spark

- Broadcast des poids modèle :

python

```
broadcast_weights = sc.broadcast(new_model.get_weights())
```

- Distribution unique des poids sur tous les workers
- Évite le chargement répétitif des poids imagenet
- Ajout critique manquant dans le notebook d'origine
- Pandas UDF Scalar Iterator :
 - Traitement par batch d'images
 - Chargement modèle une fois par worker
 - Protection contre le *Out Of Memory*
 - Réduction coûts initialisation

PCA Distribuée

- Objectif:
 - 1280 → 70 dimensions
 - Conservation maximum d'informations
 - Calcul distribué sur cluster
- Algorithme PCA:
 - Calcul distribué matrice covariance
 - Extraction composantes principales (master)
 - Transformation des features originales
 - Format Spark ML Vector en sortie
- Résultat:
 - 99.9% variance conservée
 - Facteur compression : x4.1
 - Stockage optimisé

```
k_components = 70

# Convertir les features en format ML de Spark
def to_vector(features):
    return Vectors.dense(features)

vector_udf = udf(to_vector, VectorUDT())

# Préparer les données pour la PCA
features_vector_df = features_df.withColumn("features_vector", vector_udf("f"))

# Appliquer la PCA pour réduire de 1280 à k_components dimensions
pca = PCA(k=k_components, inputCol="features_vector", outputCol="pca_features")
pca_model = pca.fit(features_vector_df)
pca_result = pca_model.transform(features_vector_df)

# Afficher la variance expliquée par les composantes principales
explained_variance = pca_model.explainedVariance
total_variance = sum(explained_variance)
print(f"Variance expliquée par les {k_components} premières composantes : {total_variance}")
print(f"Réduction de dimensionnalité : 1280 → {k_components} dimensions")

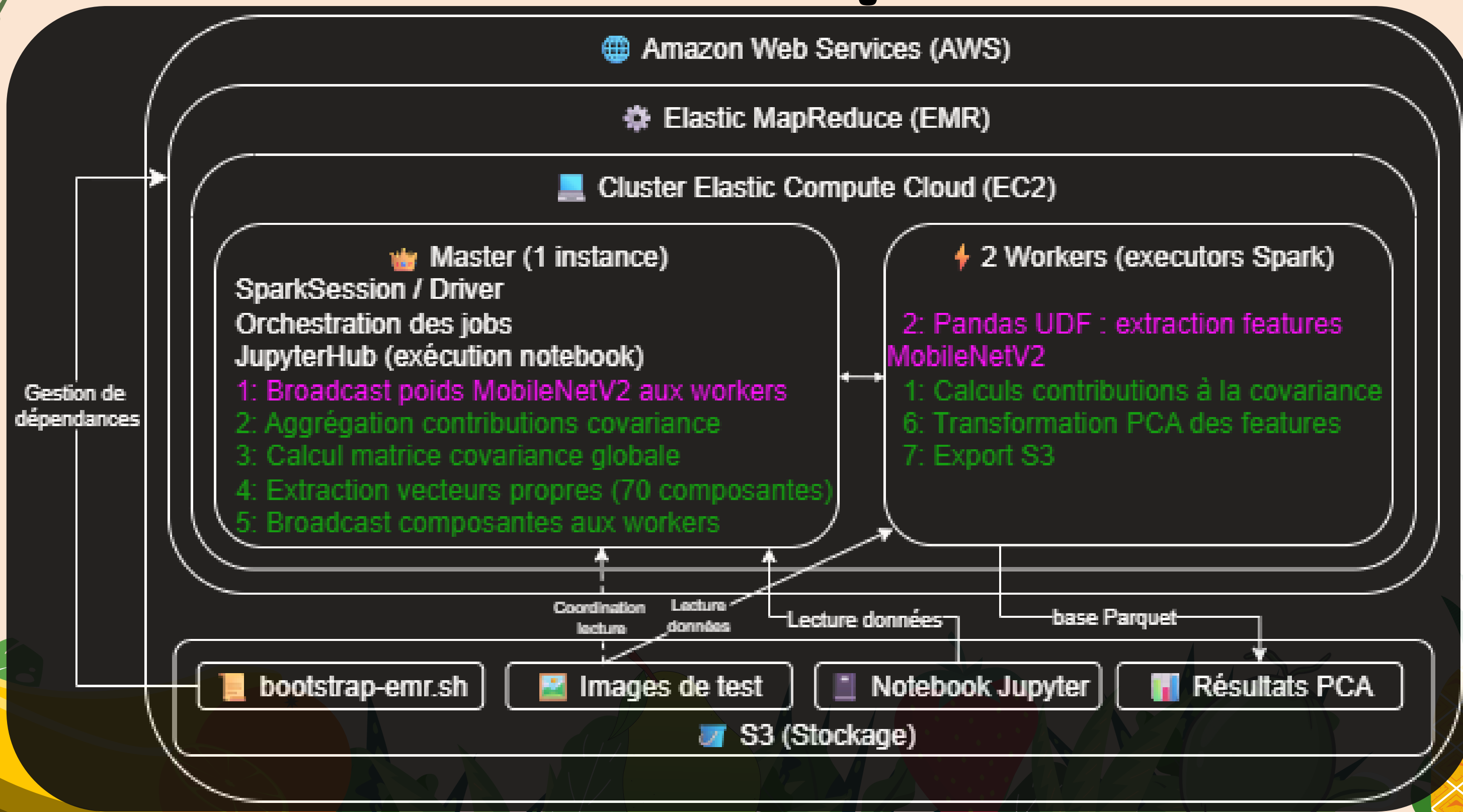
# Sélectionner les colonnes finales
final_df = pca_result.select("path", "label", "pca_features")

# Sauvegarder les résultats avec PCA sur S3
PATH_Result_PCA = PATH + '/Results_PCA'
final_df.write.mode("overwrite").parquet(PATH_Result_PCA)

print(f"Résultats sauvés sur S3 : {PATH_Result_PCA}")

FloatProgress(value=0.0, bar_style='info', description='Progress:', layout=L
Variance expliquée par les 70 premières composantes : 0.9994 (99.94%)
Réduction de dimensionnalité : 1280 → 70 dimensions
Résultats sauvés sur S3 : s3://p11-data-fruits-grgmdmn/Results_PCA
```

Schéma récapitulatif



DÉMONSTRATION



Configuration Cluster EMR

Mon cluster

Mis à jour il y a 8 minutes



Résilier

Cloner dans AWS CLI

Cloner

This EMR release reaches End of Support on Jan-24-2026 and will no longer be eligible for technical support. AWS strongly recommends that you run your workloads on the latest Amazon EMR release to receive security-critical updates and fixes. To learn more, see [Politique de support standard d'EMR](#).

▼ Récapitulatif

Informations sur le cluster

ID de cluster
j-CBZIGNM4GSP8

ARN du cluster
 arn:aws:elasticmapreduce:eu-west-3:446556758352:cluster/j-CBZIGNM4GSP8

Configuration de cluster
Groupes d'instances

Capacité
1 primaire(s) | 1 unité(s) principale(s) | 1 tâche(s)

Applications

Version d'Amazon EMR
emr-6.4.0

Applications installées
Hadoop 3.2.1, JupyterHub 1.4.1, Spark 3.1.2, TensorFlow 2.4.1

Gestion des clusters

Destination des journaux dans Amazon S3
[p11-data-fruits-grgmdmn/logs](#)

Interfaces utilisateur d'application persistantes
 [Serveur d'historique Spark](#)
 [Serveur de chronologie YARN](#)

DNS public du nœud primaire
 ec2-13-38-120-132.eu-west-3.compute.amazonaws.com
[Connexion au nœud primaire à l'aide de SSH](#)
[Connexion au nœud primaire à l'aide de SSM](#)

Statut et heure

Statut
 En attente

Heure de création
25 septembre 2025 18:07 (UTC+02:00)

Temps écoulé
47 minutes, 53 secondes

Propriétés

Actions d'amorçage

Instances (Matériel)

Étapes

Applications

Configurations

Surveillance

Évènements

identifications (1)

Journaux de cluster [Infos](#)

Archiver les fichiers journaux dans Amazon S3
Activé

Emplacement Amazon S3
[s3://p11-data-fruits-grgmdmn/logs/](#)

Chiffrement pour les journaux
Désactivé

Résiliation du cluster et remplacement des nœuds [Infos](#)

Modifier

Option de résiliation
Résilier automatiquement le cluster après le temps d'inactivité

Protection contre la résiliation
Désactivé

Temps d'inactivité
1 heure

Remplacement des nœuds défectueux
Activé

Réseau et sécurité [Infos](#)

Réseau

Cloud privé virtuel (VPC)
[vpc-07db2535ce7d2e280](#)

Sous-réseau(x) et zone(s) de disponibilité
[subnet-032ae98102dac09fc](#) | eu-west-3c

► Groupes de sécurité EC2 (pare-feu)

Configuration de sécurité

Configuration de sécurité
Aucun

Paire de clés EC2
emr-keypair-p11

Autorisations

Fonction du service pour Amazon EMR
[AmazonEMR-ServiceRole-20250923T174001](#)

Profil d'instance EC2
AmazonEMR-InstanceProfile-20250922T190128

Rôle d'autoscaling personnalisé
Non configuré

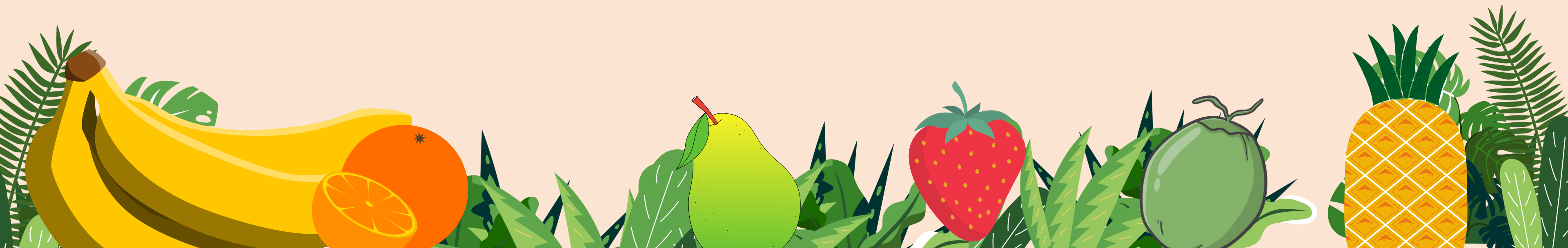
Tunneling SSH + Port Forwarding

```
(base) PS E:\NEXTCLOUD-NAS-WINDOWS\Documents\SCOLAIRE_BOULOT\Formation_openclassroom_AI_Engineer\11 Réalisez un traitement dans un environnement big data sur le cloud\notebook_etudiant> ssh -i ./emr-keypair-p11.pem -L 8890:localhost:8890 -L 9443:localhost:9443 hadoop@13.38.120.132
```

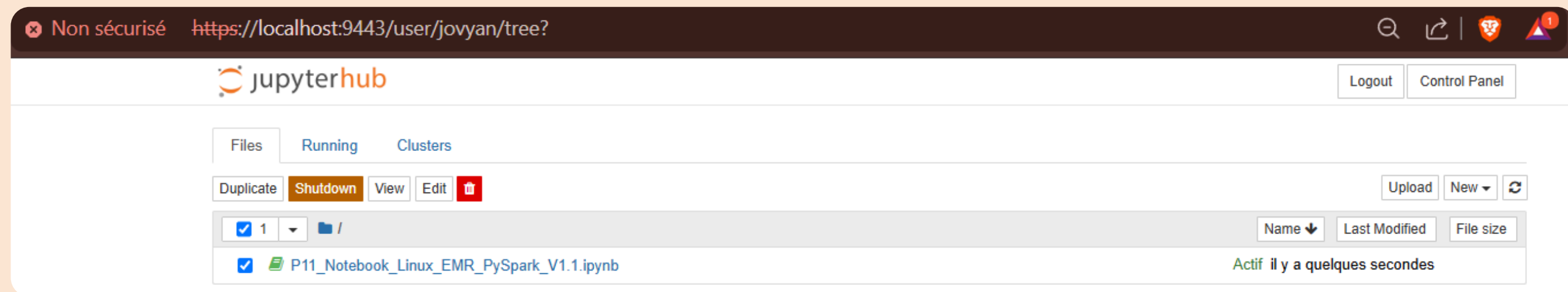
```
89 package(s) needed for security, out of 140 available  
Run "sudo yum update" to apply all updates.
```

```
EEEEEEEEEEEEEEEEEEEE MMMMMMMM          MMMMMMMM RRRRRRRRRRRRRRRR  
E::::::::::::::::::::E M::::::::M          M::::::::M R:::::::::R  
EE:::::EEEEEEEEEEE::E M::::::::M          M::::::::M R:::::RRRRRR:::R  
  E::::E      EEEEE M::::::::M          M::::::::M RR::::R      R::::R  
  E::::E      M:::::M:::M  M:::M:::::M  R:::R      R::::R  
  E:::::EEEEEEEEEEE  M:::::M M:::M M:::M M:::::M  R:::RRRRRR:::R  
  E::::::::::::::::E  M:::::M M:::M:::M M:::::M  R:::::::::RR  
  E:::::EEEEEEEEEEE  M:::::M M:::::M M:::::M  R:::RRRRRR:::R  
  E::::E      M:::::M  M:::M  M:::::M  R:::R      R::::R  
  E::::E      EEEEE M:::::M      MMM  M:::::M  R:::R      R::::R  
EE:::::EEEEEEEEEEE::E M:::::M          M:::::M  R:::R      R::::R  
E::::::::::::::::::::E M:::::M          M:::::M RR::::R      R::::R  
EEEEEEEEEEEEEEEEEEEE MMMMMMMM          MMMMMMMM RRRRRRRR      RRRRRR
```

ssh
powershell



JupyterHub (login, exécution)



JupyterHub (login, exécution)

Non sécurisé

https://localhost:9443/user/jovyan/notebooks/P11_Notebook_Linux_EMR_PySpark_V1.1.ipynb

🔍 🔗 🛡️ 🔔

jupyterhub

P11_Notebook_Linux_EMR_PySpark_V1.1

Dernière Sauvegarde : il y a 25 minutes (auto-sauvegardé)

Logout

Control Panel

File Edit View Insert Cell Kernel Widgets Help

Connexion au noyau

Fiable

PySpark

📄 + 🔍 📄 ⬆️ ⬆️ ▶️ Exécuter ⏏️ ↺️ ▶️

Markdown

4.10.5.5 Exécution d'une PCA distribuée

Limitation

Il est important de noter que la présence d'un faible échantillon (exemple minimal) va influencer sur la variance captée par chacune des composantes principales. 100 composantes principales pour un échantillon de test de 102 images va forcément mathématiquement capter presque 100% de la variance. Cela sera bien moins vrai pour un échantillon de plus de 20 000 images. La variable `k_components` sera donc probablement à ajuster, en particulier dans l'environnement cloud.

Entrée [18]:

```
k_components = 70

# Convertir Les features en format ML de Spark
def to_vector(features):
    return Vectors.dense(features)

vector_udf = udf(to_vector, VectorUDT())

# Préparer Les données pour La PCA
features_vector_df = features_df.withColumn("features_vector", vector_udf("features"))

# Appliquer La PCA pour réduire de 1280 à k_components dimensions
pca = PCA(k=k_components, inputCol="features_vector", outputCol="pca_features")
pca_model = pca.fit(features_vector_df)
pca_result = pca_model.transform(features_vector_df)

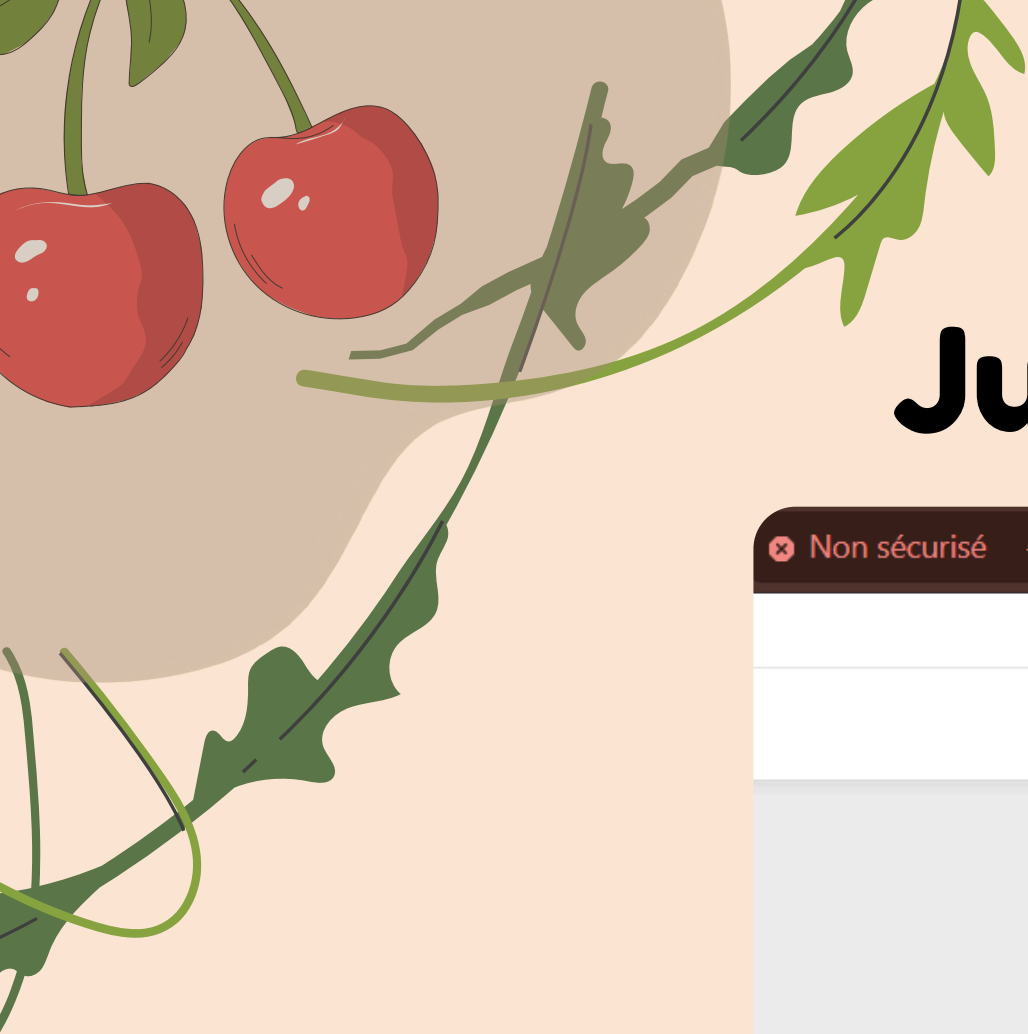
# Afficher La variance expliquée par Les composantes principales
explained_variance = pca_model.explainedVariance
total_variance = sum(explained_variance)
print(f"Variance expliquée par les {k_components} premières composantes : {total_variance:.4f} ({total_variance*100:.2f}%)")
print(f"Réduction de dimensionnalité : 1280 → {k_components} dimensions")

# Sélectionner Les colonnes finales
final_df = pca_result.select("path", "label", "pca_features")

# Sauvegarder Les résultats avec PCA sur S3
PATH_Result_PCA = PATH + '/Results_PCA'
final_df.write.mode("overwrite").parquet(PATH_Result_PCA)

print(f"Résultats sauvés sur S3 : {PATH_Result_PCA}")
```

Variance expliquée par les 70 premières composantes : nan (nan%)
Réduction de dimensionnalité : 1280 → 70 dimensions
Résultats sauvés sur S3 : s3://p11-data-fruits-grgmdmn/Results_PCA



Base Parquet

# Affichage result																			Python	
✓ 0.0s																				
	path	label	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	...	PC61	PC62	PC63	PC64	PC65	PC66	PC67		
0	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	17.432435	0.003793	0.761738	1.109528	-2.349278	-1.016772	3.785928	-0.54835	...	3.134683	-0.238344	0.284212	0.230928	-0.186516	1.199502	0.416032		
1	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	-1.560048	-0.719655	-0.390770	-1.653840	-0.992802	4.185356	5.420252	-0.54835	...	1.902873	-0.997021	-3.596620	-0.009371	-1.881502	0.054765	3.488549		
2	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	0.694892	0.707205	0.447431	-0.421446	-2.246008	-1.968085	13.842247	-0.54835	...	-0.881308	-1.033413	-0.412270	-1.289976	-1.228334	3.230407	2.828314		
3	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	10.148825	-0.196276	0.682895	-1.795817	0.361902	3.941720	7.615548	-0.54835	...	2.984482	-1.501406	-0.473990	-1.761482	-0.022067	0.209611	0.171603		
4	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	-1.052430	0.849932	-2.460750	-0.848708	0.182093	-14.506416	2.871334	-0.54835	...	1.224046	-1.413021	-0.568746	0.735982	-0.233889	-0.649144	0.840509		
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...		
97	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	19.267978	0.364912	1.332251	-0.369566	0.792905	-0.778259	5.452911	-0.54835	...	0.499446	-1.054068	-1.745435	-1.902127	0.077092	1.918598	0.732640		
98	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits	-0.734923	-1.557144	1.472118	-0.767199	-1.570988	-13.989687	2.152883	-0.54835	...	0.232537	-0.530932	0.022559	0.096099	0.403709	1.624601	1.276348		
	s3://p11-data-fruits-grgmdmn/test-multiple-fru...	test-multiple-fruits																		



Facturation

⌵ **Résumé des coûts** [Informations](#)

Coût depuis le début du mois en cours

4,16 \$US

↑ 1715913182 %par rapport au mois dernier pour la même période

Coût du mois dernier pour la même période

0,00 \$US

1-30 août

Coût total prévu pour le mois en cours

4,23 \$US

↑ 1736798733 %par rapport aux coûts totaux du mois dernier

Coût total du mois dernier

0,00 \$US

[Voir la facture](#)

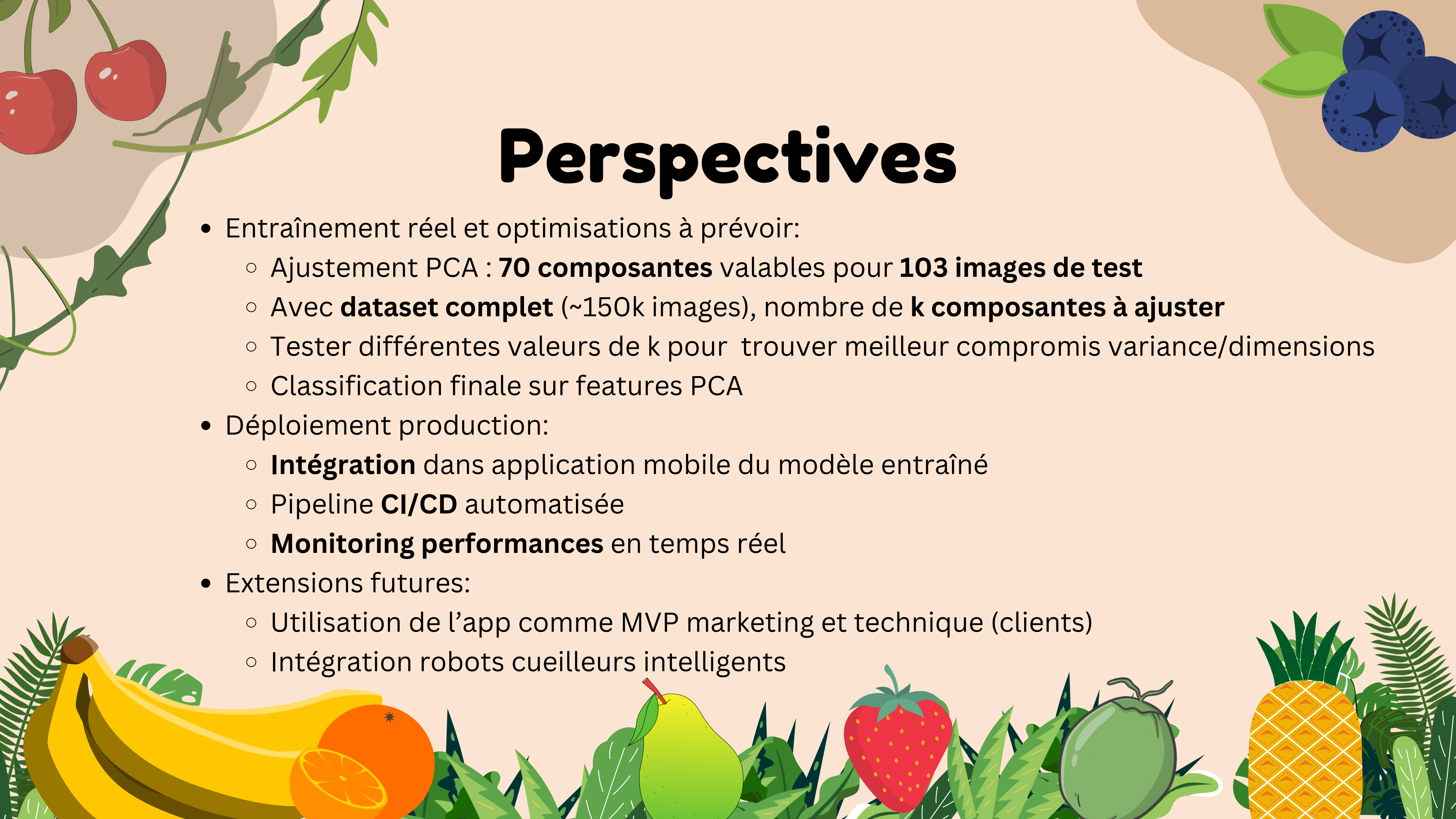
SYNTHÈSE ET CONCLUSION





Réalisations et résultats

- Chaîne de traitement complétée :
 -  Broadcast poids modèle implémenté (manquant chez l'alternant)
 -  PCA distribuée ajoutée (Spark ML)
 -  Pipeline validé local puis cloud
- Résultats techniques :
 - Réduction 1280 → 70 dimensions (PCA)
 - 99,94% variance conservée
 - Conformité RGPD (région EU-West-3)
- Architecture scalable:
 - Coût validation : < 10€ (moins d'1h de cluster)
 - Prête pour millions d'images
 - Solution managée et sécurisée



Perspectives

- Entraînement réel et optimisations à prévoir:
 - Ajustement PCA : **70 composantes** valables pour **103 images de test**
 - Avec **dataset complet** (~150k images), nombre de **k composantes à ajuster**
 - Tester différentes valeurs de k pour trouver meilleur compromis variance/dimensions
 - Classification finale sur features PCA
- Déploiement production:
 - **Intégration** dans application mobile du modèle entraîné
 - Pipeline **CI/CD** automatisée
 - **Monitoring performances** en temps réel
- Extensions futures:
 - Utilisation de l'app comme MVP marketing et technique (clients)
 - Intégration robots cueilleurs intelligents



MERCI POUR VOTRE ATTENTION!

Avez-vous des questions ?